

Handout - HIT Loop

Audi's Framework for Software Development

What to expect in this handout?

Expect guiding principles rather than a literal thing you have to follow. The software development method in this handout has worked quite well in the past 1 year after Agentic Coding became popular. You can choose your own harness.

The HIT Loop overview

The idea is to keep *humans in the loop*. Not too much, not too few. While doing so, you will also learn many new concepts thanks to the brainstorming sessions with AI. Here's an overview:

Stage	What & How	Why
Have a map	Focus on what the system should look like, not yet on the implementation details. Explore the options and various possibilities here.	Prevent the agent from solving the wrong problem, using the wrong approach, and keeping you informed with the general direction.
Install guardrails	Use behavior-driven tests to specify what must happen, while keeping how it happens flexible.	Prevent the agent from violating required behavior. Also to remind you of what's expected behavior.
Take a walk	With the map and guardrails ready, start implementation. Break the work down in Plan Mode, Coding & Tests, Code Review. Review and revise the AI's plan first, let it implement, run the behavioral tests, then review the code using your engineering judgment.	At this point, the AI can move quickly because the destination and boundaries are already clear. Let the agent execute, while the engineer continuously verifies direction and quality.

H - Have a Map

Before you ask the AI to write code, make sure both you and the AI understand **where you are going**.

The Map is the high-level direction of the system. It describes what we are building, how the major parts work together, and what important decisions need to be made.

What problems does this solve?

Coding agents are very good at writing code. But if you give them an unclear direction, they will happily write a lot of code in the wrong direction.

Common problems:

- AI starts coding before understanding the whole problem.
- Important architectural decisions are made accidentally during implementation.
- AI spends its limited context on low-level code instead of important decisions.
- Different parts of the implementation start going in different directions.
- You discover fundamental problems only after a lot of code has already been written.
- You may not understand yet what you're going to build. Do learn and discuss with AI upfront.

Surely moving fast is useless if you're moving in the wrong direction.

How does it solve the problem?

Before coding, we separate system design from implementation.

We first discuss things such as:

- What components do we need?
- What responsibility belongs where?
- What data do we need?
- How does the data flow?
- What are the important constraints?
- What can go wrong?
- What options do we have?

At this stage, avoid going too deep into code.

This allows the AI to spend its context on understanding the problem and exploring different solutions.

It also gives you an opportunity to learn.

You can discuss architecture, database design, indexes, transactions, caching, queues, trade-offs, and other engineering decisions before committing to an implementation.

How to do it?

Start by explaining the problem to the AI.

Then work together to create a high-level map.

Example:

Acceptance Criteria

- Register to the software with SSO (Google & Microsoft), and native (email and password) method.
- User to be able to login using the methods above.
- User to be able to logout using the methods above.
- Ability to invite and kick people in our organization.
- Ability to mark people as active / inactive.
- Ability to mark people as admin directly from our database.

Technical Inputs

- Focus on MVP, no need for caching, concurrency control, scalability, etc.
- But keep the back-end stateless.

"" ""

Given the requirements above, how would you do it?

Let's focus on high-level technical approach and architecture with agnostic tech stacks for now.

+ Cursor Grok 4.6 Medium



You may get response like this:

Opportunity

It is currently difficult to manage all our employees access to the system. There are employee turnovers, new hires, etc.

Worked for 1m 20s

Treat this as **identity plus org membership**, not "a login page." Tokens prove who someone is; **whether they may use the product** always comes from the database so kick, deactivate, and logout take effect immediately.

A visual breakdown is in [Org access MVP](#) — a canvas you can open beside chat. I used that instead of a long markdown dump so the data model, request path, and flows stay easy to scan.

Product rules for MVP

Invite-only. Open registration fights "invite and kick employees." An email gets an invite; they complete signup with Google, Microsoft, or password. Seed the first admin in the database (same as "mark as admin from SQL").

Single organization. One company, one org row. Keep `organizations` + `memberships` so multi-org later is additive.

Admin is not an API. Role lives in `memberships.role`. The app reads it; ops updates it in SQL. UI can show admin, but must not promote.

Architecture

```

Client → Stateless API → Database (source of truth)
      ↳ Google OIDC / Microsoft OIDC
      ↳ Email (invite links only)
  
```

No session store, no sticky sessions. Any API instance can handle any request: verify a signed JWT locally, then **load user + membership**. That DB read is required for MVP — otherwise inactive/kicked users stay valid until the token expires.

Data (minimum)

Table	Purpose
users	Person: unique email, optional password_hash, token_version
identities	SSO link: {provider, ↓ ovider_subject} → user

Org Access Mvp Architecture

High-level approach for employee signup, login, logout, and org membership. Stack-agnostic. Backend stays stateless; the database is the only shared state.

Invite-only
Join path

OIDC + password
Auth methods

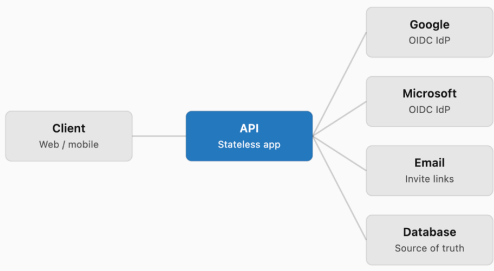
DB on every request
AuthZ check

Core split

Treat this as two products glued together: authentication (who are you) and organization membership (are you allowed in, and with which role). Login tokens prove identity. Membership, active/inactive, and admin are always read from the database so kick and deactivate take effect immediately.

System shape

One API process, no session store, no sticky sessions. Google and Microsoft are identity providers only — they never own org roles.



```

graph LR
    Client[Client  
Web / mobile] --- API[API  
Stateless app]
    API --- Google[Google  
OIDC IdP]
    API --- Microsoft[Microsoft  
OIDC IdP]
    API --- Email[Email  
Invite links]
    API --- Database[Database  
Source of truth]
  
```

Recommended product rules

Decision	MVP choice	Why
----------	------------	-----

Do not get overwhelmed, do not get intimidated.

Give the AI inputs, and if you're overwhelmed, breakdown and ask clarifying question like below:

My input:

- We will use the Authorization Code + PKCE flow for the SSO integrations.
- Let's use email for now for the stable IdP.

Some clarifying question:

- From what you described, this means your authorization approach will read DB on every request instead of just validating the signature of the JWT?

+ Grok 4.6 Medium ▾



After back and forth conversation, do agree on a direction (use your judgement) and dig dive on that direction if you have doubts like below:

Layout the general approach for the Authorization Code + PKCE flow.

+ Grok 4.6 Medium ▾



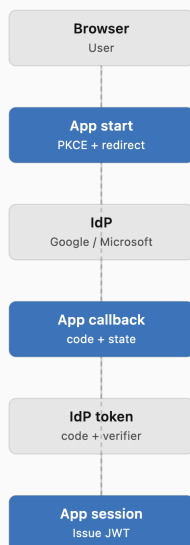
Authorization Code + PKCE

General SSO approach for Google and Microsoft. The app is a confidential client: it holds the client secret, still uses PKCE, and keeps verifier/state in a short-lived cookie so the API stays stateless.

What PKCE adds

The authorization code is useless without the code_verifier that only the party who started the flow has. Even with a client secret, PKCE stops a stolen callback code from being exchanged by someone else.

Round trips



Stateless handshake cookie

On start, the API mints an HttpOnly, Secure, SameSite=Lax cookie (few minutes TTL), signed or encrypted. No server session store.

Field	Why
provider	google microsoft
state	CSRF. Callback state must match.
nonce	Must appear in the ID token.
code_verifier	Sent only to the token endpoint.
invite_token?	If this SSO is completing an invite.

It is recommended to also align optimizations early on like follows:

Excellent. We have decided to use a relational database.
What's your planned query access patterns?
What's your planned database indexes?

+ Grok 4.6 Medium ▾



Query access patterns

Pattern	When	How you find the row
AuthZ	Every authenticated request	<code>users.id + memberships (user_id, org_id)</code>
Native login	Email/password	<code>users.email</code> , then the same membership join
SSO callback	After token exchange	<code>identities (provider, email)</code> ; if missing, <code>users.email</code> to link
Consume invite	Register	<code>invites.token_hash</code> (pending, not expired)
Create invite	Admin	Pending invite <code>(org_id, email)</code> ; also <code>users.email</code> so you do not invite someone already in
List members	Admin	<code>memberships.org_id</code> , join <code>users</code>
Kick / active-inactive	Admin	Same unique as AuthZ: <code>(user_id, org_id)</code>
Logout	Logout	<code>UPDATE users.token_version WHERE id = :sub</code>

Indexes

Uniques you already want; they **are** the indexes for login and AuthZ. Only two extras: `memberships(org_id)` and `identities(user_id)`.

Table	Index	Why
<code>users</code>	PRIMARY KEY (<code>id</code>)	AuthZ, logout
<code>users</code>	UNIQUE (<code>email</code>)	Native login, invite, SSO link
<code>identities</code>	UNIQUE (<code>provider, email</code>)	SSO upsert
<code>identities</code>	INDEX (<code>user_id</code>)	FK; optional for MVP
<code>organizations</code>	PRIMARY KEY (<code>id</code>)	FK only
<code>memberships</code>	UNIQUE (<code>user_id, org_id</code>)	AuthZ / kick / status. B-tree left prefix covers <code>user_id</code>
<code>memberships</code>	INDEX (<code>org_id</code>)	List members — <code>org_id</code> is not leftmost on the unique key
<code>invites</code>	UNIQUE (<code>token_hash</code>)	Open invite link
<code>invites</code>	Partial UNIQUE (<code>org_id, email</code>) <code>WHERE consumed_at IS NULL</code>	One pending invite per email

You can still challenge the proposed optimizations like below:

I think indexing `invites` on the pending invite per email is too much.

+ Cursor Grok 4.6 Medium 🔒



Rinse and repeat until you have a clear map. How do you know you already have a clear map? use your judgement. But a general rule of thumb is you know the components: you know the database design, you know the optimizations, etc.

Best Practices

Start technologically agnostic

Do not immediately ask: "How should I implement this using Go, PostgreSQL, and Redis?"

I suggest starting with: "What does this system need to do, and what components do we actually need?" Technology comes later.

Discuss options, not just answers

Do not ask AI only for one solution.

Ask:

- "What are our options?"
- "What are the trade-offs?"
- "When would option A be better than option B?"

The goal is to use AI as a thinking partner, not as a magician.

Focus on important decisions

Do not design every function and every file.

The Map should focus on decisions that are expensive to change later:

- architecture,
- system boundaries,
- data ownership,
- database design,
- consistency,
- major failure modes.

Leave small implementation decisions for later.

Challenge the AI

Do not accept its first recommendation automatically.

Ask:

- "Why?"
- "What could go wrong?"
- "What is the simpler alternative?"
- "What assumptions are we making?"

AI proposes. **You decide.**

I - Install Guardrails

Once you know where you are going, define the boundaries. Guardrails are the behaviors that must stay correct while the AI is writing code.

The goal is to give the agent freedom on how to implement, but be strict on what must happen.

What problems does this solve?

Coding agents can move very fast. But fast execution creates another problem: they can also introduce wrong behavior very quickly.

Common problems:

- AI misunderstands an important requirement.
- AI fixes one thing but breaks another.
- AI chooses an implementation that technically works, but violates business rules.
- Refactoring changes behavior accidentally.
- You only discover mistakes during manual testing or code review.

A prompt alone is not enough. You can tell the AI: "An order confirmation ID must not be reused.". However surely you know it will be forgotten 5 months later. The stronger version is to make that requirement executable.

How does it solve the problem?

We turn important behaviors into tests before letting the agent implement them.

For example:

"""

Given an order confirmation ID has already been used

When another order tries to use the same ID

Then the operation must fail

And the existing order must remain unchanged

"""

Now the agent has a clear boundary.

It is still free to decide:

- which function to create,
- which query to use,
- how the repository works,
- whether to refactor something,
- how the packages are structured.

But the required behavior must continue to work. This is why behavior-driven tests work well as guardrails.

They define **what the system must do**, without unnecessarily defining **how the system must do it**.

How to do it?

Start from the important behaviors in your Map.

Ask:

- What must always happen?
- What must never happen?
- What happens when something fails?
- What edge cases are important?
- What existing behavior must not break?
- Others (use your judgement).

Then turn those answers into tests.

For example, suppose the requirement is: "A user can only confirm an order once."

Write the behavior first:

""

Given an unconfirmed order

When the user confirms the order

Then the order becomes confirmed

""

Then add the important failure case:

""

Given an already confirmed order

When the user tries to confirm it again

Then the operation must fail

""

Then let the agent implement the solution.

If the tests fail, the agent is outside the guardrails.

Here's a full example:

""

Excellent. Now we need the guardrails.

I prefer a behavior-driven tests, where we do not test the implementation detail too thoroughly (overfitting).

Some of the tests I need to enforce:

1. Reject login when the password is incorrect.
2. A user are allowed to join an org if their invite link is not expired
3. A user are rejected to join an org if their invite link is expired.
4. User will automatically links their existing account if they attempt to register with different auth provider.
5. User with admin privilege can kick other user in the same organization.
6. User without admin privilege cannot kick other user in the same organization.
7. User with admin privilege cannot kick other user in different organization.
8. User with admin privilege cannot kick themselves.

We need to enforce it on the service-layer's level. Mocking the repository layers is fine.

Suggest more tests.

You may get response like this, where the AI suggests more:

Suggest more (same style)

Auth — correct password + active membership succeeds (pair for #1). Unknown email rejected. Inactive membership rejected. No membership (kicked) rejected. SSO-only user cannot native-login (null password). Logout: previous token version fails `authorize()` if that lives on the service.

Invite / join — unknown token rejected. Already-consumed invite rejected. Authenticating email ≠ invite email rejected. No invite rejected (closed org). Valid join creates `role=member`, `status=active` (never admin). Only an admin **of that org** can create an invite (member cannot; other-org admin cannot).

Linking — same provider + email does not create a second user. Different emails do not merge. Unverified IdP email does not link or create. Linking does not change role or status.

Org — admin can set another member inactive/active in the same org; non-admin cannot; other-org admin cannot. Kick of someone already gone: pick reject vs no-op and lock it. **Role is data:** flip `role` to admin in the fake repo and the same actor can kick — that enforces “admin from the database” without testing SQL.

Best Practices

Test behavior, not implementation

Avoid tests like

`OrderService` must call `OrderRepository.Save()` once.

This locks the implementation.

Prefer:

When an order is confirmed,
the confirmation must be persisted.

The first test says **how**. The second test says **what**. Keep the how flexible.

Focus on important behavior

Do not try to test every possible detail.

Prioritize:

- business rules,
- critical flows,
- failure cases,
- data integrity,
- security-sensitive behavior,
- behavior that would be expensive to break.

Guardrails should protect what matters most.

Include negative cases

Do not only test the happy path.

Do also ask "what should never be allowed?"

For example:

""""

A payment cannot be captured twice.

A user cannot access another user's order.

Inventory cannot become negative.

An already-used confirmation ID cannot be reused.

""""

Negative cases often make the strongest guardrails.

Do not let the agent quietly change the guardrails

If implementation becomes difficult, the easiest solution for an agent may be to change the test.

Be careful.

A failing test usually means one of two things:

1. The implementation is wrong.
2. The expected behavior itself needs to change.

The agent should not decide this alone.

If a guardrail needs to change, **you decide why it changes**.

Guardrails do not replace code review

Passing tests means: **The system behaves as we specified.**

It does not mean: **The implementation is good.**

The code may still contain:

- bad abstractions,
- inefficient queries,
- race conditions,
- poor naming,
- security issues,
- unnecessary complexity.

Guardrails protect behavior.

Engineering judgment still protects quality

T - Take a Walk

Once you have the Map and the Guardrails, it is time to let the agent work.

But do not just say: "Implement this."

Take the work through three stages:

- Plan Mode
- Coding & Tests
- Code Review

The idea is simply to let the agent move fast, but keep human checkpoints at the important decisions.

What problems does this solve?

Coding agents are very good at execution.

But if you give them too much freedom too early, several things can happen:

- The task becomes too large.
- The agent makes wrong assumptions.
- The implementation slowly drifts away from the original Map.
- The agent introduces unnecessary complexity.
- The code works, but the engineering quality is poor.
- You only notice problems after a large amount of code has already been written.

The problem is not that AI cannot write code.

The problem is: **AI can write a lot of wrong code very quickly.**

So we need a controlled way to let it execute.

How does it solve the problem?

The idea is as follows.

Plan Mode: Are we going in the right direction in this specific task?

Coding & Tests: Does it actually behave correctly?

Code Review: Is this good engineering?

This separates direction, correctness, and quality.

How to do it?

This time, open your harness. Then, describe the context (map), the acceptance criteria (guardrails), and some input (based on the map too). Example below.

```
> # Context
We're making functionality for people to register on this app.

# Acceptance Criteria & Tests
Let's start with Google-based OIDC authentication
- User to be able to register with Google, using Authorization Code + PKCE.
- A successful authentication with Google is when the ID token is verified.
- User not registered are detected by whether user already exists in the first place or not.
- When user has registered, return access token, treat it as user login.

# Technical Inputs
- The callback endpoint shall be "/v1/auth/google/callback"
- No refresh token mechanism for now.

# Tests
Have tests at the service (usecase) layer, focus on behaviors (acceptance criteria) rather than implementation details.█
```

Grok 4.7 (high) · plan

Enter:send | Opt+Enter:newline | Shift+Tab:mode | Ctrl+x:shortcuts

The AI harness will probably ask you clarifying questions. Do answer them with your human judgement.

When Google authentication succeeds (ID token verified) but no user exists yet, what should the callback do?

- ☐ **Register and log in (Recommended)** Create the user from the verified Google identity, then return an access token. If the user already exists, skip creation and return an access token for that user. Both outcomes are a logged-in session; tests tell them apart by whether a user was created.
- ☐ **Detect only, no access token** Do not create a user. Return a distinct not-registered result and no...
- ☐ **Type your answer here**

[1/2] ↑/↓ navigate · +/- question · y copy Enter:select

Tab:next answer | Esc:scrollback | Shift+x:dismiss

Finally the AI will give you the overall plan. **Do not just accept it.** Do read it, do review it.

```

plan.md
1 Google registration (Authorization Code + PKCE)
2
3 People register and log in through one callback. The client runs the Google redirect, then POST /v1/auth/google/callback with the authorization code, the PKCE code_verifier, and the redirect_uri. The backend exchanges the code (confidential client, using the existing Google client secret), verifies the ID token, and either creates the user or recognizes an existing one. Both outcomes return an access token. There is no refresh token and no authorize/start endpoint.
4
5 account_tab stays the chart of accounts. A person is a new user record, looked up by Google's sub.
6
7 Flow
8
9
10 mermaid: sequenceDiagram
sequenceDiagram
    participant Client
    participant API as POST /v1/auth/google/callback
    participant Google
    participant DB as user_tab

    Client->>API: code, code_verifier, redirect_uri
    API->>Google: token exchange with code_verifier and client secret
    Google-->>API: ID token
    API->>API: verify ID token
    API->>DB: find user by Google sub
    alt no user
        API->>DB: insert user
    else user exists
        API->>DB: do not insert
    
```

a approve | s request changes | c comment | y copy plan | q quit plan

Waiting on plan approval

When in doubt, do attack it.

- There will be plan to use microsoft auth in the future. I disagree with putting 'google_sub' inside of 'user_tab'. Use 'identity_tab' table instead of merging it inside of 'user_tab'. The 'identity_tab' should contain 'provider', and 'provider_subject'. User will be able to link multiple auth providers in the future. Make sure the current infrastructure can support that.

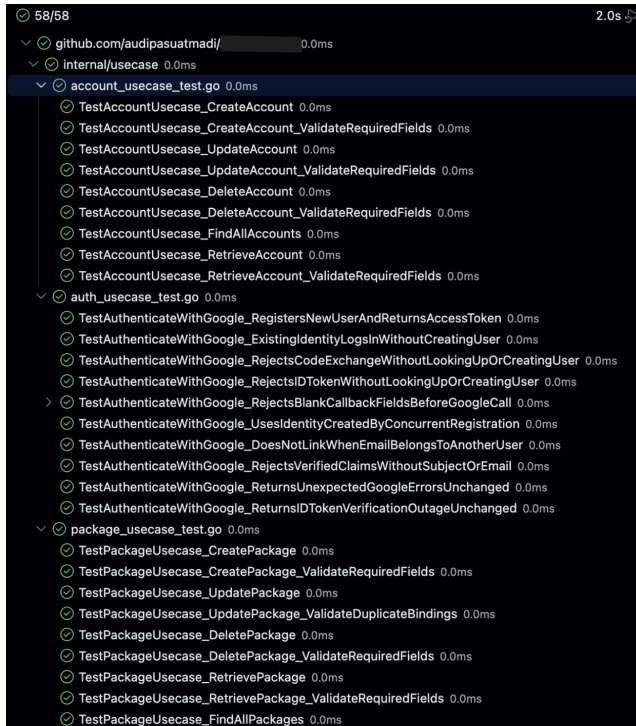
Grok 4.7 (high) · commenting L108

Enter:save comment | Esc:cancel

It is better to have it clear upfront rather than having a problem too late.

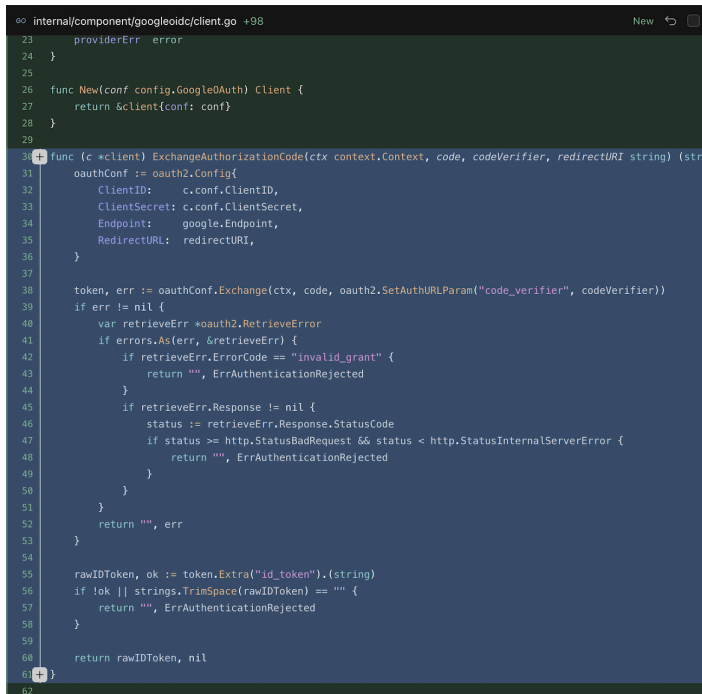
When you think it's appropriate, finally approve it.

What to do after coding is done? first thing first, run your tests like below:



This way, you will catch regressions far earlier. Since your tests are behavior-driven tests, you can quickly align the business expectations early.

Finally in the end, do code reviews. **This is important.** AI can generate things like N+1 query, suboptimal pattern, no observability, and even an outright mess in your code.



Rinse and repeat, and don't forget to have fun.

Best Practices

Keep each walk small

Do not give the agent the whole Map at once.

Break it into manageable pieces.

Instead of:

"Implement the payment system."

Prefer:

"Implement payment creation."

Then:

"Implement payment confirmation."

Then:

"Implement refund handling."

Smaller walks are easier to understand, review, and correct.

Spend time in Plan Mode

Do not rush this stage.

Fixing a bad plan is cheap.

Fixing a large implementation based on a bad plan is expensive.

A few minutes spent reviewing the plan can prevent a lot of unnecessary code.

Do not blindly accept AI recommendations

Plan Mode is not an approval screen. Do read the plan, challenge assumptions, and ask why. You still own the engineering decision.

Run the Guardrails before reviewing style

First:

Does it behave correctly?

Then:

Is it implemented well?

This keeps the review focused.

Review the code as if a human wrote it

Do not lower your standards because the code came from AI. Ask the same questions you would ask in a normal pull request. Would you approve this code if it came from another engineer? If not, do not approve it from an AI agent either.

Give feedback, not just fixes

When you find a problem, explain what should happen instead.

For example:

"Do not call the repository repeatedly here. Load the data in one query."

This helps the agent revise the implementation while keeping the reasoning visible.



Audi Pasuatmadi
[linkedin.com/in/audipasuatmadi/](https://www.linkedin.com/in/audipasuatmadi/)

Keep going back to the Map and Guardrails

When you are unsure about a decision:

Go back to the **Map**.

When you are unsure whether behavior is correct:

Go back to the **Guardrails**.

They are your reference points throughout implementation.